

ASTRAIOS

Technical Whitepaper

Quantitative ML Trading Platform · v1.0 · May 2026

Astraios is a full-stack quantitative trading platform for cryptocurrency derivatives. It combines a Transformer-based signal engine, real-time Bybit integration for USDT perpetuals, and a React/FastAPI web application with per-user API key isolation, live/paper mode toggle, and a 1-second position update loop.

1. System Architecture

Astraios follows a monorepo structure with a Python FastAPI backend, a React single-page application, a PostgreSQL database, and a separate ML pipeline for training on AWS SageMaker. The platform serves both an API and the built frontend from a single uvicorn process in production.

Layer	Technology	Role
API Server	FastAPI + Uvicorn	REST endpoints, SPA serving, lifespan hooks
Database	PostgreSQL + asyncpg	Users, signals, positions (async SQLAlchemy)
Background Jobs	asyncio tasks	Signal refresh (15 min), price refresh (5 min)
Market Data	Bybit API + yfinance	Klines, tickers, price feeds
ML Engine	PyTorch + SageMaker	MarketTransformer training and inference
Frontend	React 19 + Vite	SPA: dashboard, charts, trade terminal
Charts	lightweight-charts v4	Candlestick + volume, 1s poll
Auth	JWT (python-jose)	Per-user auth, 24h expiry
Migrations	Alembic	Schema versioning

1.1 Data Flow

At startup, the lifespan hook creates all database tables and launches two background asyncio tasks. Signal generation runs every 15 minutes: it fetches the latest 100 candles per symbol from Bybit, engineers 25 features, runs the trained MarketTransformer, and writes one signal per ticker per user to PostgreSQL. Price refresh runs every 5 minutes via yfinance for any open paper positions. On the frontend, Bybit positions and wallet data poll the live API every 1 second; chart candles update every 5 seconds.

2. MarketTransformer — Signal Engine

The signal engine uses a Transformer encoder architecture trained to classify the forward 6-hour return of a cryptocurrency futures contract into three classes: BUY ($>+1\%$), SELL ($<-1\%$), or HOLD ($\pm 1\%$). The model was trained on AWS SageMaker and achieves 66.1% validation accuracy across 12 symbols — versus a 33% random baseline for a balanced 3-class problem.

2.1 Architecture

Component	Configuration
Input projection	Linear(25 $\rightarrow$ 128)
Positional encoding	Sinusoidal, max_len=512
Encoder layers	3 $\times$ TransformerEncoderLayer

Attention heads	4
d_model	128
Feed-forward dim	256
Activation	GELU
Dropout	0.1 (training), 0.0 (inference)
Classification head	Linear(128→256) → GELU → Dropout → Linear(256→3)
Output	3 logits → softmax probabilities

2.2 Input Sequence

The model operates on a sliding window of 32 consecutive 1-hour candles (32 hours of history). Each candle is represented as a 25-dimensional feature vector. The input tensor shape is (batch, 32, 25). The final encoder token (position -1) is passed to the classification head.

3. Feature Engineering

All 25 features are computed from raw OHLCV kline data. Features are normalised using a StandardScaler fitted on the training corpus; the scaler mean and scale vectors are serialised into config.json alongside the model weights for consistent inference.

Feature	Formula / Description	Category
returns	pct_change(close)	Return
log_returns	log(close / close[-1])	Return
ema_ratio_8	close / EMA(close, 8)	Trend
ema_ratio_21	close / EMA(close, 21)	Trend
ema_ratio_50	close / EMA(close, 50)	Trend
ema_cross_8_21	EMA(8) – EMA(21)	Trend
rsi_14	RSI(14)	Oscillator
rsi_7	RSI(7)	Oscillator
macd	EMA(12) – EMA(26)	Momentum
macd_signal	EMA(macd, 9)	Momentum
macd_hist	macd – macd_signal	Momentum
bb_pct	(close – BB_lower) / (BB_upper – BB_lower)	Volatility
atr_norm	ATR(14) / close	Volatility
vol_ratio	volume / MA(volume, 20)	Volume
ret_lag_1..5	returns shifted 1, 2, 3, 5 bars	Lag
mom_5/10/20	close / close[-N] – 1	Momentum

high_low_range	(high – low) / close	Range
close_position	(close – low) / (high – low)	Range
rolling_vol_5	std(returns, 5)	Volatility
rolling_vol_20	std(returns, 20)	Volatility

4. Training Procedure

4.1 Dataset

Training data is collected via the Bybit linear kline endpoint. For each of 11 symbols, 1,000 1-hour candles are fetched (paginated in batches of 200), features are engineered, and labels are generated from 6-bar forward returns. The final dataset contains 10,714 samples with the following class balance: SELL 2,108 (19.7%), HOLD 6,429 (60.0%), BUY 2,177 (20.3%).

4.2 Label Generation

- Forward horizon: 6 candles (6 hours on 1h chart)
- BUY (class 2): forward_return > +1%
- SELL (class 0): forward_return < –1%
- HOLD (class 1): $-1\% \leq \text{forward_return} \leq +1\%$

4.3 Training Configuration

Hyperparameter	Value
Epochs	50
Batch size	64
Optimizer	AdamW (lr=0.001, weight_decay=1e-4)
LR schedule	Cosine Annealing (T_max=50)
Loss function	CrossEntropyLoss with inverse-frequency class weights
Gradient clipping	1.0 (max norm)
Train/val split	80/20 per symbol (chronological, no leakage)
Training platform	AWS SageMaker ml.m5.xlarge (CPU)
Best val accuracy	66.1%

4.4 Class Weighting

To address class imbalance (HOLD accounts for 60% of samples), class weights are computed as the inverse of class frequency, then normalised to sum to 3. This prevents the model from collapsing to always predicting HOLD.

5. Inference Pipeline

At signal generation time, the model singleton is loaded once into memory from `ml/output/model.pt` and `ml/output/config.json`. For each symbol, the following steps run synchronously inside `asyncio.to_thread` to avoid blocking the event loop:

- Fetch 100 most recent 1h candles from Bybit (proxied if US region)
- Extract close, high, low, volume arrays
- Engineer 25 features using the same pipeline as training
- Replace NaN/inf with 0, normalise with stored scaler parameters
- Slice the last 32 timesteps into the input sequence
- Forward pass → softmax → argmax for action, max probability for confidence
- Append RSI context (overbought/oversold) to rationale string
- Fall back to heuristic momentum scoring if model files are absent

6. REST API Reference

6.1 Authentication

All endpoints except `/api/health` require a Bearer JWT token. Tokens are issued on POST `/api/auth/register` and POST `/api/auth/login`, expire after 24 hours, and carry the user UUID as the subject claim.

6.2 Trade Endpoints

Method	Path	Description
GET	<code>/api/trade/klines</code>	Kline data (up to 1,000 candles, paginated)
GET	<code>/api/trade/symbols</code>	All USDT perp symbols sorted by 24h volume
GET	<code>/api/trade/positions</code>	Open positions (live or testnet)
GET	<code>/api/trade/wallet</code>	Account equity, available balance, PnL
GET	<code>/api/trade/orders</code>	Open orders
POST	<code>/api/trade/order</code>	Place market order with optional TP/SL
POST	<code>/api/trade/close</code>	Close position with reverse market order
POST	<code>/api/trade/leverage</code>	Set symbol leverage on Bybit

6.3 Signal Endpoints

Method	Path	Description
GET	<code>/api/signals</code>	Latest signal per USDT ticker, sorted by confidence DESC
POST	<code>/api/signals</code>	Manually create a signal
DELETE	<code>/api/signals/{id}</code>	Delete a signal

6.4 Account Endpoints

Method	Path	Description
GET	/api/account/stats	Portfolio metrics and key configuration status
POST	/api/account/api-keys	Save mainnet Bybit API credentials
DELETE	/api/account/api-keys	Remove mainnet credentials
POST	/api/account/testnet-keys	Save testnet Bybit API credentials
DELETE	/api/account/testnet-keys	Remove testnet credentials

7. Bybit Integration

All Bybit calls use the `pybit.unified_trading.HTTP` client with the linear (USDT perpetual) category. Authentication is per-user: each API call resolves the user's stored mainnet or testnet credentials from PostgreSQL, falling back to server-level environment variables if present. An optional HTTP/HTTPS proxy is configured via `BYBIT_PROXY` in `.env` to route around regional IP restrictions.

- Market orders only — no limit order support in this release
- GTC (Good Till Cancel) time-in-force for all orders
- Unified account mode required (supports cross-margin USDT perps)
- Kline pagination: 200 candles per request, looped to fetch up to 1,000

8. Database Schema

8.1 Users

Column	Type	Notes
id	UUID	Primary key, default uuid4
email	VARCHAR(320)	Unique, indexed
hashed_password	VARCHAR(128)	bcrypt
name	VARCHAR(120)	
plan	VARCHAR(40)	Default 'Private Beta'
is_active	BOOLEAN	Default true
created_at	TIMESTAMPTZ	UTC
bybit_api_key	VARCHAR(256)	Nullable, mainnet
bybit_api_secret	VARCHAR(256)	Nullable, mainnet
bybit_testnet_key	VARCHAR(256)	Nullable, testnet
bybit_testnet_secret	VARCHAR(256)	Nullable, testnet

8.2 Signals

Column	Type	Notes
id	UUID	Primary key
user_id	UUID	FK → users, indexed
ticker	VARCHAR(20)	Indexed, USDT perp symbol
action	VARCHAR(10)	'BUY', 'SELL', 'HOLD'
confidence	FLOAT	[0.0, 1.0]
rationale	TEXT	Nullable
created_at	TIMESTAMPTZ	UTC

8.3 Positions

Column	Type	Notes
id	UUID	Primary key
user_id	UUID	FK → users, indexed
ticker	VARCHAR(20)	Indexed
quantity	FLOAT	
entry_price	FLOAT	
current_price	FLOAT	Updated by price refresh job
created_at	TIMESTAMPTZ	UTC
updated_at	TIMESTAMPTZ	Auto-updated on write

9. Background Scheduler

Two asyncio tasks are launched on application startup via the FastAPI lifespan context manager. Both tasks run inside an infinite loop with `asyncio.sleep` between iterations.

Task	Interval	Action
refresh_signals	Every 15 minutes	Generate transformer signals for all users
refresh_prices	Every 5 minutes	Update current_price on paper positions via yfinance

10. Security Model

- Passwords hashed with bcrypt (passlib)
- JWTs signed with HS256; secret rotatable via JWT_SECRET env var
- API keys stored in plaintext in PostgreSQL — encryption at rest recommended for production

- Each trade API call resolves credentials from the authenticated user's DB row — cross-user key access is impossible by design
- CORS configured for all origins in development; should be restricted to the frontend domain in production
- .env excluded from version control via .gitignore

11. AWS SageMaker Training Pipeline

The ML pipeline is decoupled from the main application. Training is triggered manually via `ml/launch_sagemaker.py`, which packages the training script, uploads the dataset CSV to S3, and submits a SageMaker `CreateTrainingJob` request using the `boto3` client directly (bypassing the SageMaker SDK to avoid version compatibility issues).

Parameter	Value
Training image	pytorch-training:2.1.0-cpu-py310-ubuntu20.04-sagemaker
Instance type	ml.m5.xlarge
Max runtime	3,600 seconds
S3 bucket	astraiosbucket
Output path	s3://astraiosbucket/ml/training/output/
Model artifacts	model.pt + config.json → model.tar.gz

12. Frontend Architecture

The React SPA is built with Vite 6 and served directly from the FastAPI process via `StaticFiles` mounts. The application uses `React Router` for client-side routing across three views: landing (`/`), login (`/login`), and dashboard (`/dashboard`).

- React 19, React Router DOM, pure CSS (no UI framework)
- `lightweight-charts v4` for candlestick charts with 1s live polling
- Trading mode (live/paper), selected symbol, and chart interval persisted in `localStorage`
- Bybit symbol selector fetches all 546 USDT perps from the API, searchable
- Dual render pattern: desktop table + mobile card stack for positions and signals
- Responsive breakpoints at 980px (tablet) and 640px (mobile)

13. Supported Signal Symbols

The signal engine covers 12 high-liquidity USDT perpetual futures on Bybit. Symbols were selected based on 24h turnover, market maturity, and data availability going back at least 1,000 hours.

Symbol	Underlying	Category
BTCUSDT	Bitcoin	Layer 1
ETHUSDT	Ethereum	Layer 1
SOLUSDT	Solana	Layer 1
SUIUSDT	Sui	Layer 1
DOGEUSDT	Dogecoin	Meme
XRPUSDT	XRP	Layer 1
LINKUSDT	Chainlink	Oracle
AAVEUSDT	Aave	DeFi
AVAXUSDT	Avalanche	Layer 1
ARBUSDT	Arbitrum	Layer 2
WIFUSDT	dogwifhat	Meme
1000PEPEUSDT	PEPE ×1000	Meme

Astraios · Technical Whitepaper · v1.0 · May 2026

This document describes the technical architecture of the Astraios platform as of May 2026. Astraios is a technology platform — not investment advice. Cryptocurrency derivatives trading carries significant risk of loss. Past signal accuracy does not guarantee future performance.